

Anisa Aulia Alhaqi/18224080

Soal 1 - Circular Buffer

- a. constant IDX_UNDEF : integer = -1
constant CAPACITY : integer = 100
type ElType : < id : integer,
cost : integer >
type queue : < buffer : array[0..CAPACITY-1] of ElType,
idxHead : integer,
idxTail : integer >
- b. function isFull (q : queue) → boolean
→ ((q.idxHead = 0 and q.idxTail = CAPACITY-1) or (q.idxHead = q.idxTail+1 and q.idxHead > q.idxTail))
- c. procedure enqueue (input/output q : queue, input val : ElType)
if(not isFull(q)) then
 if(isEmpty(q)) then
 q↑.idxHead ← 0
 q↑.idxTail ← 0
 else
 q↑.idxTail ← (q↑.idxTail+1) mod CAPACITY
 q↑.buffer[q↑.idxTail] ← val
- d. procedure dequeue (input/output q : queue, output val : ElType)
 if(not isEmpty(q)) then
 val ← q↑.buffer[q↑.idxHead]
 if(q↑.idxHead = q↑.idxTail) then
 q↑.idxHead ← IDX_UNDEF
 q↑.idxTail ← IDX_UNDEF
 else
 q↑.idxHead ← (q↑.idxHead+1) mod CAPACITY

Soal 2 - Round Robin

procedure roundRobin (input/output q : Queue, input t : integer)

KAMUS LOKAL

e : ElType

ALGORITMA

dequeue(q, e)

if(q↑.buffer[q↑.idxHead].cost > t) then
 output("<", val.id, "> telah selesai diproses")

else

enqueue(q, val↑.cost-t)

Soal 3 - Priority Queue

ADT

konstanta CAPACITY = 100

konstanta IDX_UNDEF = -1

Tipe ElType: < id: integer,
 cost: integer >

Tipe Queue: < buffer: Array[0..CAPACITY-1] of ElType,
 idxHead: integer, // Indeks elemen pertama dalam queue
 idxTail: integer > // Indeks elemen terakhir dalam
 queue

procedure enqueue (input/output q : Queue, input val: ElType)

{Algoritma ini dibuat untuk kondisi q.buffer yang sudah terurut}

KAMUS LOKAL

 i : integer

ALGORITMA

depend on (q) :

 (q↑.idxHead = q↑.idxTail) :

if(q↑.idxHead = -1) then

 q↑.idxHead ← 0

 q↑.idxTail ← 0

 q↑.buffer[0] ← val

else

if(q↑.buffer[0] > val) then

if(q↑.idxHead = 0) then

 q↑.buffer[1] ← q↑.buffer[0]

 q↑.buffer[0] ← val

 q↑.idxTail ← q↑.idxTail+1

else

 q↑.idxTail ← q↑.idxTail+1

 q↑.buffer[q↑.idxTail] ← val

 (q↑.buffer[0].cost > val.cost): {saat insert first}

if(q↑.idxHead = 0) then

```

    q↑.idxTail ← q↑.idxTail+1
    j traversal[q↑.idxTail...1]
    q↑.buffer[j] ← q↑.buffer[j-1]
    q↑.buffer[0] ← val
else
    q↑.idxHead ← q↑.idxHead-1
    q↑.buffer[q↑.idxHead] ← val
(q↑.idxHead ≠ q↑.idxTail):
    if(q↑.idxHead ≠ 0) then
        i traversal[q↑.idxHead...q↑.idxTail]
        q↑.buffer[i-q↑.idxHead] = q↑.buffer[i]
        q↑.idxTail ← q↑.idxTail-q↑.idxHead+1
        q↑.idxHead ← 0
    i ← q↑.idxHead
    while(i < q↑.idxTail and not(q↑.buffer[i].cost ≤ val.cost
and q↑.buffer[i+1].cost ≥ val.cost)) do
        i ← i+1
    if(i = q↑.idxTail) then {artinya harus dimasukin ke yg habis
idxTail, tapi kalau ini ga harus geser, tinggal masukin aja}
        q↑.idxTail ← q↑.idxTail+1
        q↑.buffer[q↑.idxTail] ← val
    else {input di tengah}
        q↑.idxTail ← q↑.idxTail+1
        j traversal[q↑.idxTail...i+2]
        q↑.buffer[j] ← q↑.buffer[j-1]
        q↑.buffer[i+1] ← val

```

procedure dequeue (input/output q : Queue, output val : ElType)

KAMUS LOKAL

i : integer

ALGORITMA

```

val ← q↑.buffer[q↑.idxHead]
q↑.idxHead ← q↑.idxHead+1

```